

TASK 1 TEAM 4 - HƯỚNG DẪN CHI TIẾT CHO NGƯỜI 2

Transform + Object/Bin State + Planner/FSM

Tài liệu nội bộ của BA x AILab

Hạng mục	Nội dung
Phiên bản	v1.0 FULL
Đối tượng	Thư/Bảo+Đoàn/Huy+Viễn/Công: trong nhóm 4 sinh viên cùng thực hiện Task 1 - Precise Desktop Sorting
Vai trò	Transform, chuẩn hóa object/bin state, lập kế hoạch gấp-đặt, thiết kế FSM và retry logic
Mục tiêu cuối	Biến output perception thành ActionPlan rõ ràng để Người 3 thực thi pick-place và Người 4 đánh giá được
PM hỗ trợ	PM hỗ trợ baseline/config/repo/run script; Người 2 không làm thay PM nhưng phải giữ logic kỹ thuật của Task 1
Output chính	transform_utils.py, bins.yaml, object_bin_pose_report.json, planner.py, task1_fsm.yaml, action_plan_schema.json, retry_policy.yaml, planner_trace.jsonl

Thông điệp chính: Người 2 là cầu nối giữa Perception và Motion. Nếu Người 1 phát hiện đúng vật nhưng Người 2 map sai bin, chọn sai object, hiểu sai frame hoặc FSM thiếu VERIFY/RETRY, Task 1 vẫn thất bại. Mục tiêu không phải viết nhiều code, mà là tạo logic task rõ, kiểm thử được và khó gây nhầm lẫn.

Mục lục định hướng

1. Vai trò của Người 2 trong Task 1
2. Người 2 cần tự học module nào
3. Input/output contract của Người 2
4. Cấu trúc thư mục và artifact cần tạo
5. Step-by-step triển khai từ ngày đầu
6. Transform sanity và object/bin state
7. Planner: chọn object, chọn bin, tạo ActionPlan
8. FSM Task 1 và retry/failure logic
9. Mock testing trước khi nối simulation thật
10. Kế hoạch 5 ngày cho Người 2
11. Phối hợp với Người 1, Người 3, Người 4 và PM
12. Debug cookbook cho Planner/FSM

- 13. Definition of Done và gói nộp cá nhân
- 14. Rubric 100 điểm
- 15. Phụ lục code skeleton

1. Vai trò của Người 2 trong Task 1

Người 2 chịu trách nhiệm trả lời câu hỏi: từ danh sách vật thể do Người 1 xuất ra, robot nên gấp vật nào trước, đặt vào bin nào, theo state nào, và nếu lỗi thì xử lý thế nào?

```
Perception output từ Người 1
-> validate object state
-> kiểm tra frame/pose/confidence
-> map class_id sang bin
-> chọn vật tiếp theo
-> tạo ActionPlan
-> gọi Motion primitive của Người 3
-> nhận PrimitiveResult
-> chuyển sang VERIFY/RETRY/DONE/FAIL
```

Người 2 KHÔNG làm chính	Người 2 PHẢI làm chính
Không tự viết detector thay Người 1	Định nghĩa object state nào hợp lệ để planner dùng
Không tune gripper/waypoint thay Người 3	Tạo ActionPlan đầy đủ cho pick-place
Không viết toàn bộ evaluator thay Người 4	Bảo đảm mỗi state và mỗi failure có log được
Không làm thay PM phần baseline/config/repo	Kiểm tra logic Task 1 không phụ thuộc hard-code mơ hồ

2. Người 2 cần tự học module nào

Thứ tự	Module	Mức độ	Cần lấy được gì
1	Module 0 - Đề thi, chiến lược, guilds	Đọc nhanh	Hiểu success condition của Task 1, metric success/time/accuracy và pipeline hybrid
2	Module 2 - Isaac Sim/USD/URDF/Walker S2/Baseline	Đủ dùng	Hiểu object path, bin path, camera path, config Task 1 do PM cung cấp
3	Module 3 - Robotics Math	Học sâu phần frame/SE(3)	Hiểu T_base_camera, pose_base, quaternion/yaw, kiểm tra frame sanity
4	Module 4 - Camera/RGB-D/Transform	Đủ dùng	Hiểu output của Người 1: centroid_camera_m, pose_base, depth issues
5	Module 6 - Motion Primitives	Đủ dùng	Biết Người 3 cần object_pose_base, bin_pose_base, grasp_hint, retry_policy
6	Module 9 - Task-by-Task Solution Design	Học sâu Task 1	Thiết kế FSM, task contract, failure handling
7	Module 10 - Evaluation/Logging/Ablation	Đủ dùng	Ghi planner_trace, failure_reason, metrics cho Người 4

3. Input/output contract của Người 2

3.1. Input từ Người 1 - Perception

Người 2 nhận danh sách object state. Không giả định dữ liệu luôn đúng. Bước đầu tiên của planner là validate object state.

```
{
  "frame_id": 25,
  "timestamp": 12.53,
  "camera_name": "front_rgb",
  "objects": [
    {
      "object_id": "obj_001",
      "class_id": "part_A",
      "confidence": 0.91,
      "bbox_xyxy": [310, 220, 380, 292],
      "centroid_px": [345.0, 256.0],
      "centroid_camera_m": [0.12, -0.04, 0.74],
      "pose_base": {
        "position_m": [0.46, -0.15, 0.82],
        "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
      },
      "grasp_hint": {
        "yaw_rad": 1.57,
        "grasp_width_m": 0.045,
        "approach_axis": "z_down"
      },
      "failure_reason": null
    }
  ]
}
```

3.2. Output sang Người 3 - Motion

Output quan trọng nhất của Người 2 là ActionPlan. Người 3 không nên phải tự suy luận class nào vào bin nào hoặc tự chọn object.

```
{
  "plan_id": "plan_007",
  "state": "PLAN_PICK_PLACE",
  "primitive": "pick_place",
  "object_id": "obj_001",
  "class_id": "part_A",
  "object_pose_base": {
    "position_m": [0.46, -0.15, 0.82],
    "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
  },
  "target_bin": "bin_A",
  "bin_pose_base": {
    "position_m": [0.55, 0.20, 0.80],
    "quaternion_xyzw": [0.0, 0.0, 0.0, 1.0]
  },
  "grasp_hint": {
    "yaw_rad": 1.57,
    "grasp_width_m": 0.045,
    "approach_axis": "z_down"
  },
  "retry_policy": "retry_once_slow",
  "timeout_s": 20.0
}
```

3.3. Output sang Người 4 - Evaluation

```
{
  "event": "planner_decision",
  "episode_id": "ep_003",
```

```

"step_id": 5,
"fsm_state": "SELECT_OBJECT",
"selected_object_id": "obj_001",
"selected_class_id": "part_A",
"selected_bin": "bin_A",
"object_confidence": 0.91,
"planner_score": 0.88,
"decision_reason": "highest_confidence_reachable",
"failure_reason": null
}

```

4. Cấu trúc thư mục và artifact cần tạo

```

person2_planner_pack/
  README_PLANNER.md
  src/task1/
    transform_utils.py
    state.py
    planner.py
    fsm.py
    planner_debug.py
  configs/
    bins.yaml
    planner.yaml
    retry_policy.yaml
  schemas/
    action_plan_schema.json
    object_state_schema.json
    primitive_result_schema.json
  outputs/
    pytest_transform.txt
    object_bin_pose_report.json
    planner_trace.jsonl
    task1_fsm.yaml
    action_plan_examples.jsonl
    planner_unit_test_report.txt
    failure_cases_planner.jsonl

```

5. Step-by-step triển khai từ ngày đầu

Step 1 - Nhận config tối thiểu từ PM và Người 1

Thông tin cần nhận	Nguồn	Dùng để làm gì
Tên class vật	Người 1/PM	Map part_A/part_B sang bin
Bin pose hoặc bin path	PM	Tạo bins.yaml
Camera/base transform convention	Người 1/PM	Kiểm tra pose_base hợp lệ
Reachable workspace sơ bộ	Người 3/PM	Không chọn object ngoài vùng với tới
Primitive input format	Người 3	Tạo ActionPlan đúng schema
Log schema	Người 4	Ghi planner_trace dùng được cho evaluation

```

# outputs/task1_planner_start_checklist.md
- Perception class names: part_A, part_B
- Bin names: bin_A, bin_B
- Bin pose source: configs/bins.yaml or PM scene map

```

- Object pose frame: robot_base
- Confidence threshold: 0.70
- Motion primitive name: pick_place
- Retry policy default: retry_once_slow
- Open questions:
 1. ...

Step 2 - Tạo bins.yaml

Người 2 phải chuẩn hóa bin map. Không để logic map class sang bin nằm rải rác trong code.

```
# configs/bins.yaml
class_to_bin:
  part_A: bin_A
  part_B: bin_B

bins:
  bin_A:
    pose_base:
      position_m: [0.55, 0.20, 0.80]
      quaternion_xyzw: [0.0, 0.0, 0.0, 1.0]
      safe_place_offset_m: [0.0, 0.0, 0.05]
      margin_m: 0.03
  bin_B:
    pose_base:
      position_m: [0.55, -0.20, 0.80]
      quaternion_xyzw: [0.0, 0.0, 0.0, 1.0]
      safe_place_offset_m: [0.0, 0.0, 0.05]
      margin_m: 0.03
```

Step 3 - Viết object state validator

Trước khi chọn object, planner phải loại object không đủ thông tin để tránh motion nhận pose rác.

```
def validate_object_state(obj, min_confidence=0.70):
    required = ["object_id", "class_id", "confidence", "pose_base", "grasp_hint"]
    for key in required:
        if key not in obj:
            return False, f"MISSING_{key.upper()}"

    if obj.get("failure_reason"):
        return False, obj["failure_reason"]

    if obj["confidence"] < min_confidence:
        return False, "LOW_CONFIDENCE"

    pos = obj["pose_base"].get("position_m")
    quat = obj["pose_base"].get("quaternion_xyzw")
    if pos is None or len(pos) != 3:
        return False, "INVALID_POSITION"
    if quat is None or len(quat) != 4:
        return False, "INVALID_QUATERNION"

    if any(not isinstance(v, (int, float)) for v in pos):
        return False, "NON_NUMERIC_POSITION"

    return True, None
```

Step 4 - Kiểm tra object pose có hợp lý không

Sanity check	Gợi ý
z_base của object gần mặt bàn/object height	Nếu z âm hoặc quá cao, có thể sai depth hoặc transform
x/y nằm trong vùng thao tác	Nếu object ở xa ngoài workspace, không chọn
quaternion có norm gần 1	Nếu norm quá lệch, normalize hoặc reject
class_id có trong class_to_bin	Nếu không, set failure_reason CLASS_NOT_MAPPED

```
import math

def quaternion_norm(q):
    return math.sqrt(sum(float(x) * float(x) for x in q))

def is_pose_sane(obj, workspace):
    x, y, z = obj["pose_base"]["position_m"]
    q = obj["pose_base"]["quaternion_xyzw"]
    if abs(quaternion_norm(q) - 1.0) > 0.05:
        return False, "BAD_QUATERNION_NORM"
    if not (workspace["x_min"] <= x <= workspace["x_max"]):
        return False, "X_OUT_OF_WORKSPACE"
    if not (workspace["y_min"] <= y <= workspace["y_max"]):
        return False, "Y_OUT_OF_WORKSPACE"
    if not (workspace["z_min"] <= z <= workspace["z_max"]):
        return False, "Z_OUT_OF_WORKSPACE"
    return True, None
```

Step 5 - Chọn object tiếp theo

Chiến lược MVP: chọn object hợp lệ, chưa xử lý, confidence cao, pose trong workspace. Không tối ưu quá sớm.

```
def score_object(obj):
    # Điểm đơn giản: confidence cao và gần vùng thao tác trung tâm hơn.
    conf = float(obj["confidence"])
    x, y, z = obj["pose_base"]["position_m"]
    lateral_penalty = 0.05 * abs(y)
    height_penalty = 0.02 * abs(z)
    return conf - lateral_penalty - height_penalty

def select_next_object(objects, handled_ids, workspace, min_confidence=0.70):
    candidates = []
    rejected = []

    for obj in objects:
        oid = obj.get("object_id", "unknown")
        if oid in handled_ids:
            rejected.append((oid, "ALREADY_HANDLED"))
            continue

        ok, reason = validate_object_state(obj, min_confidence=min_confidence)
        if not ok:
            rejected.append((oid, reason))
            continue

        ok, reason = is_pose_sane(obj, workspace)
        if not ok:
            rejected.append((oid, reason))
            continue

        candidates.append((score_object(obj), obj))
```

```

if not candidates:
    return None, rejected

candidates.sort(key=lambda x: x[0], reverse=True)
return candidates[0][1], rejected

```

Step 6 - Map class sang bin

```

def map_class_to_bin(class_id, class_to_bin):
    if class_id not in class_to_bin:
        raise ValueError(f"Class not mapped to bin: {class_id}")
    return class_to_bin[class_id]

```

Step 7 - Tạo ActionPlan

```

def make_action_plan(obj, bins_config, retry_policy="retry_once_slow"):
    class_to_bin = bins_config["class_to_bin"]
    bin_name = map_class_to_bin(obj["class_id"], class_to_bin)
    bin_info = bins_config["bins"][bin_name]

    return {
        "primitive": "pick_place",
        "object_id": obj["object_id"],
        "class_id": obj["class_id"],
        "object_pose_base": obj["pose_base"],
        "target_bin": bin_name,
        "bin_pose_base": bin_info["pose_base"],
        "grasp_hint": obj["grasp_hint"],
        "retry_policy": retry_policy,
        "timeout_s": 20.0
    }

```

6. Transform sanity và object/bin state

Người 2 không nhất thiết tự tính toàn bộ camera-to-base, nhưng phải đủ hiểu để phát hiện các lỗi frame gây gấp lệch hoặc đặt sai bin.

File	Mục đích	Nội dung tối thiểu
object_bin_pose_report.json	So sánh object pose và bin pose trong cùng frame	object_id, class_id, position_m, target_bin, bin_position_m, distance_to_bin
pytest_transform.txt	Kết quả unit test transform/pose validator	pass/fail và lỗi cụ thể
transform_sanity_notes.md	Ghi chú lỗi frame nghi ngờ	T_base_camera convention, unit, quaternion order

```

# outputs/object_bin_pose_report.json
{
  "frame": "robot_base",
  "objects": [
    {
      "object_id": "obj_001",
      "class_id": "part_A",
      "object_position_m": [0.46, -0.15, 0.82],
      "target_bin": "bin_A",

```

```

    "bin_position_m": [0.55, 0.20, 0.80],
    "pose_sanity": "ok"
  }
],
"warnings": []
}

```

7. Planner: chọn object, chọn bin, tạo ActionPlan

Nguyên tắc Planner: Planner không được gọi motion nếu object không hợp lệ, class chưa map sang bin, bin pose chưa có, hoặc object nằm ngoài workspace. Fail sớm bằng failure_reason rõ ràng tốt hơn là để robot chạy sai.

Tình huống	Planner nên làm gì
Không có object hợp lệ	Chuyển state sang REOBSERVE hoặc FAIL_NO_VALID_OBJECT
Object confidence thấp	Không chọn; ghi LOW_CONFIDENCE
Class không thuộc part_A/part_B	Không chọn; ghi CLASS_NOT_MAPPED
Object ngoài workspace	Không chọn; ghi OUT_OF_WORKSPACE
Motion trả IK_FAIL	Retry nếu còn lượt; nếu không chọn object khác hoặc fail
Verify sai bin	Ghi WRONG_BIN, không mark handled
Verify đúng	Add object_id vào handled_ids

8. FSM Task 1 và retry/failure logic

FSM là phần quan trọng nhất của Người 2. Không có FSM rõ ràng thì Task 1 dễ biến thành code spaghetti và rất khó debug.

```

RESET
-> OBSERVE
-> SELECT_OBJECT
-> PLAN_PICK_PLACE
-> EXECUTE_PICK_PLACE
-> VERIFY
-> NEXT_OBJECT nếu handled_count < 4
-> DONE nếu handled_count == 4
-> RETRY nếu fail có thể sửa
-> FAIL nếu timeout hoặc lỗi nghiêm trọng

```

State	Input	Action	Output/Next
RESET	episode start	clear handled_ids, retry_count, logs	OBSERVE
OBSERVE	RGB-D/perception call	nhận object list từ Người 1	SELECT_OBJECT hoặc FAIL_NO_OBJECT
SELECT_OBJECT	object list	validate + score + chọn object	PLAN_PICK_PLACE hoặc REOBSERVE
PLAN_PICK_PLACE	selected object	map class to bin + tạo ActionPlan	EXECUTE_PICK_PLACE hoặc FAIL_PLAN
EXECUTE_PICK_PLACE	ActionPlan	gọi Người 3 motion primitive	VERIFY hoặc RETRY/FAIL_MOTION
VERIFY	PrimitiveResult + observation	gọi Người 4 verify	NEXT_OBJECT/DONE/RETRY/FAIL_VERIFY
RETRY	failure reason + retry count	giảm tốc, tăng pregrasp height, reobserve	PLAN hoặc SELECT hoặc FAIL
DONE	handled_count == 4	ghi task_success	end

State	Input	Action	Output/Next
FAIL	fatal error/timeout	ghi failure_reason	end

8.1. task1_fsm.yaml mẫu

```

states:
  RESET:
    next: OBSERVE
  OBSERVE:
    on_success: SELECT_OBJECT
    on_no_object: FAIL
  SELECT_OBJECT:
    on_success: PLAN_PICK_PLACE
    on_low_confidence: OBSERVE
    on_no_valid_object: FAIL
  PLAN_PICK_PLACE:
    on_success: EXECUTE_PICK_PLACE
    on_out_of_workspace: SELECT_OBJECT
    on_missing_bin: FAIL
  EXECUTE_PICK_PLACE:
    on_success: VERIFY
    on_ik_fail: RETRY
    on_drop: RETRY
    on_collision: FAIL
  VERIFY:
    on_correct_bin: NEXT_OBJECT
    on_wrong_bin: RETRY
    on_object_missing: RETRY
  NEXT_OBJECT:
    condition_done: DONE
    condition_remaining: OBSERVE
  RETRY:
    max_retries_per_object: 1
    on_retry_available: OBSERVE
    on_retry_exhausted: SELECT_OBJECT
  DONE: {}
  FAIL: {}

```

8.2. Retry policy mẫu

```

# configs/retry_policy.yaml
retry_once_slow:
  max_retries_per_object: 1
  changes:
    pre_grasp_height_delta_m: 0.03
    descend_speed_scale: 0.7
    grasp_wait_extra_s: 0.5
  retry_on:
    - IK_FAIL
    - DROP
    - VERIFY_OBJECT_MISSING
  no_retry_on:
    - CLASS_NOT_MAPPED
    - SEVERE_COLLISION
    - CONFIG_ERROR

```

9. Mock testing trước khi nổi simulation thật

Người 2 có thể làm được nhiều việc mà không cần chờ Isaac Sim chạy. Hãy tạo mock objects và mock primitive result để kiểm tra planner/FSM sớm.

```
MOCK_OBJECTS = [
    {
        "object_id": "obj_001",
        "class_id": "part_A",
        "confidence": 0.92,
        "pose_base": {"position_m": [0.45, -0.10, 0.82], "quaternion_xyzw": [0,0,0,1]},
        "grasp_hint": {"yaw_rad": 0.0, "grasp_width_m": 0.045, "approach_axis": "z_down"},
        "failure_reason": None,
    },
    {
        "object_id": "obj_002",
        "class_id": "part_B",
        "confidence": 0.88,
        "pose_base": {"position_m": [0.47, 0.12, 0.82], "quaternion_xyzw": [0,0,0,1]},
        "grasp_hint": {"yaw_rad": 1.57, "grasp_width_m": 0.045, "approach_axis": "z_down"},
        "failure_reason": None,
    },
]

def test_select_and_plan():
    handled_ids = set()
    obj, rejected = select_next_object(MOCK_OBJECTS, handled_ids, WORKSPACE)
    assert obj is not None
    plan = make_action_plan(obj, BINS_CONFIG)
    assert plan["primitive"] == "pick_place"
    assert plan["target_bin"] in ["bin_A", "bin_B"]
```

Test case	Kỳ vọng
4 object hợp lệ	Planner chọn object confidence cao nhất và tạo ActionPlan
Object confidence thấp	Reject LOW_CONFIDENCE
Object class lạ	Reject CLASS_NOT_MAPPED
Object ngoài workspace	Reject OUT_OF_WORKSPACE
handled_ids chứa obj_001	Không chọn lại obj_001
Motion fail IK_FAIL lần 1	Chuyển RETRY
Motion fail IK_FAIL lần 2	Chọn object khác hoặc FAIL theo policy

10. Kế hoạch 5 ngày cho Người 2

Ngày	Mục tiêu	Việc chính	Output bắt buộc
Ngày 1	Nắm contract và config	Nhận checklist từ PM, đọc output Người 1, tạo bins.yaml và schema	task1_planner_start_checklist.md, bins.yaml, action_plan_schema.json
Ngày 2	Validate object và transform sanity	Viết object validator, pose sanity, class-to-bin mapping	transform_utils.py, object_bin_pose_report.json, pytest_transform.txt
Ngày 3	Planner MVP	Chọn object, tạo ActionPlan, mock test	planner.py, action_plan_examples.jsonl, planner_unit_test_report.txt
Ngày 4	FSM MVP	Viết task1_fsm.yaml, retry_policy.yaml, planner_trace	task1_fsm.yaml, retry_policy.yaml, planner_trace.jsonl

Ngày	Mục tiêu	Việc chính	Output bắt buộc
Ngày 5	Tích hợp với nhóm	Nối output Người 1 và Người 3, ghi log cho Người 4, sửa failure top 3	first_planner_integration_report.md, failure_cases_planner.jsonl

11. Phối hợp với Người 1, Người 3, Người 4 và PM

Đối tác	Người 2 cần nhận	Người 2 cần cung cấp
Người 1 - Perception	perception_interface.json, pose_base, confidence, grasp_hint	schema feedback, rejected reasons, confidence threshold
Người 3 - Motion	primitive input requirements, failure codes, workspace hints	ActionPlan, bin_pose_base, retry_policy
Người 4 - Evaluation	log schema, verify result, failure taxonomy	planner_trace.jsonl, decision_reason, failure_reason
PM	scene/config/bin path, run environment, merge support	task1_fsm.yaml, interface changes, blocker list

Quy tắc giao tiếp: Nếu Người 2 đổi tên field trong ActionPlan hoặc thay logic class-to-bin, phải báo ngay Người 1, Người 3, Người 4 và PM. Lỗi interface thường làm mất nhiều thời gian hơn lỗi thuật toán.

12. Debug cookbook cho Planner/FSM

Lỗi	Triệu chứng	Cách kiểm tra	Cách sửa ưu tiên
Map sai bin	Vật đúng class nhưng đi sai bin	Kiểm tra bins.yaml và planner_trace	Sửa class_to_bin, thêm unit test
Chọn lại vật đã xử lý	Robot gặp lại object cũ hoặc vùng trống	Kiểm tra handled_ids và verify result	Chỉ add handled khi verify success; reobserve sau place
Object pose ngoài workspace vẫn được chọn	Motion báo IK_FAIL nhiều	In object_bin_pose_report	Thêm workspace filter trước make_action_plan
FSM kẹt ở OBSERVE	Không chuyển state dù có object	Kiểm tra failure_reason từ Người 1	Log rejected objects; giảm threshold tạm thời nếu hợp lý
FSM không retry	Fail một lần là dừng	Kiểm tra retry_count và retry_policy	Thêm RETRY state và max_retries_per_object
Retry vô hạn	Episode timeout	Kiểm tra retry_count reset sai	Giới hạn retry theo object_id và episode
ActionPlan thiếu field	Motion crash KeyError	Validate action_plan_schema	Thêm schema check trước gọi motion
Failure unknown quá nhiều	Evaluation không phân tích được	Kiểm tra planner_trace	Chuẩn hóa failure_reason enum

13. Definition of Done và gói nộp cá nhân

Điều kiện	Bắt buộc
Có bins.yaml với class_to_bin rõ ràng	Có
Có object state validator	Có
Có workspace/pose sanity check	Có
Có select_next_object không chọn object đã handled	Có
Có make_action_plan đúng schema	Có
Có task1_fsm.yaml	Có

Điều kiện	Bắt buộc
Có retry_policy.yaml	Có
Có planner_trace.jsonl	Có
Có mock tests pass	Có
Người 3 chạy được motion từ ActionPlan	Có
Người 4 đọc được planner decision log	Có

```

person2_planner_pack/
  README_PLANNER.md
  src/task1/transform_utils.py
  src/task1/state.py
  src/task1/planner.py
  src/task1/fsm.py
  configs/bins.yaml
  configs/planner.yaml
  configs/retry_policy.yaml
  schemas/action_plan_schema.json
  outputs/object_bin_pose_report.json
  outputs/planner_trace.jsonl
  outputs/action_plan_examples.jsonl
  outputs/planner_unit_test_report.txt
  outputs/failure_cases_planner.jsonl

```

14. Rubric chấm phần Người 2 - 100 điểm

Hạng mục	Điểm	Tiêu chí
Config/class-to-bin/bin pose	15	bins.yaml rõ ràng, không hard-code rải rác, map part_A/B đúng
Validate object state và transform sanity	15	Loại được object thiếu field, confidence thấp, pose/quaternion/workspace bất thường
Planner chọn object	15	Không chọn object đã handled, có scoring, có rejected reasons
ActionPlan schema	15	Đủ object_pose_base, bin_pose_base, grasp_hint, retry_policy, timeout
FSM Task 1	20	Có đầy đủ OBSERVE, SELECT, PLAN, EXECUTE, VERIFY, RETRY, DONE/FAIL
Mock tests và planner trace	10	Có unit test, planner_trace, action_plan_examples
Phối hợp nhóm và debug	10	Interface ổn định, failure_reason rõ, hỗ trợ Người 1/3/4 debug nhanh

15. Phụ lục code skeleton

15.1. planner.py skeleton

```

class Task1Planner:
    def __init__(self, bins_config, workspace, min_confidence=0.70):
        self.bins_config = bins_config
        self.workspace = workspace
        self.min_confidence = min_confidence
        self.handled_ids = set()

```

```

def reset(self):
    self.handled_ids.clear()

def mark_handled(self, object_id):
    self.handled_ids.add(object_id)

def plan(self, perception_output):
    objects = perception_output.get("objects", [])
    obj, rejected = select_next_object(
        objects,
        handled_ids=self.handled_ids,
        workspace=self.workspace,
        min_confidence=self.min_confidence,
    )
    if obj is None:
        return None, {
            "failure_reason": "NO_VALID_OBJECT",
            "rejected": rejected,
        }

    try:
        action_plan = make_action_plan(obj, self.bins_config)
    except Exception as e:
        return None, {
            "failure_reason": "PLAN_ERROR",
            "error": str(e),
            "selected_object": obj.get("object_id"),
        }

    return action_plan, {
        "failure_reason": None,
        "selected_object": obj.get("object_id"),
        "rejected": rejected,
    }

```

15.2. fsm.py skeleton

```

from enum import Enum

class Task1State(str, Enum):
    RESET = "RESET"
    OBSERVE = "OBSERVE"
    SELECT_OBJECT = "SELECT_OBJECT"
    PLAN_PICK_PLACE = "PLAN_PICK_PLACE"
    EXECUTE_PICK_PLACE = "EXECUTE_PICK_PLACE"
    VERIFY = "VERIFY"
    RETRY = "RETRY"
    NEXT_OBJECT = "NEXT_OBJECT"
    DONE = "DONE"
    FAIL = "FAIL"

class Task1FSM:
    def __init__(self):
        self.state = Task1State.RESET
        self.retry_count_by_object = {}

    def transition(self, event):
        # Keep this explicit for beginners; do not hide state changes.
        if self.state == Task1State.RESET:
            self.state = Task1State.OBSERVE
        elif self.state == Task1State.OBSERVE:

```

Tài liệu tự học nội bộ - dùng cho training camp HRC2026

```

        self.state = Task1State.SELECT_OBJECT if event == "objects_ready" else Task1State.FAIL
    elif self.state == Task1State.SELECT_OBJECT:
        self.state = Task1State.PLAN_PICK_PLACE if event == "object_selected" else Task1State.OBSERVE
    elif self.state == Task1State.PLAN_PICK_PLACE:
        self.state = Task1State.EXECUTE_PICK_PLACE if event == "plan_ready" else Task1State.FAIL
    elif self.state == Task1State.EXECUTE_PICK_PLACE:
        self.state = Task1State.VERIFY if event == "primitive_success" else Task1State.RETRY
    elif self.state == Task1State.VERIFY:
        if event == "verify_success_all_done":
            self.state = Task1State.DONE
        elif event == "verify_success_remaining":
            self.state = Task1State.OBSERVE
        else:
            self.state = Task1State.RETRY
    elif self.state == Task1State.RETRY:
        self.state = Task1State.OBSERVE if event == "retry_available" else Task1State.FAIL
    return self.state

```

15.3. planner_trace logger

```

import json
from datetime import datetime

def log_planner_event(path, **kwargs):
    record = {
        "time": datetime.utcnow().isoformat() + "Z",
        **kwargs,
    }
    with open(path, "a", encoding="utf-8") as f:
        f.write(json.dumps(record, ensure_ascii=False) + "
")

```

Kết luận cho Người 2

Người 2 hoàn thành tốt khi biến được perception output thành ActionPlan ổn định, có FSM rõ ràng, có retry/failure logic và có planner_trace để cả nhóm debug. Đây là vai trò giữ nhịp logic của Task 1: Perception có thể nhìn đúng, Motion có thể gấp tốt, nhưng nếu Planner/FSM sai thì hệ thống vẫn thất bại.